PORTFOLIO

김준우 Joon Woo Kim

DevOps / Cloud Engineer

VFX 제작 환경의 운영·자동화 경험을 Kubernetes, CI/CD, GitOps 기반 Cloud Native 환경으로 확장하고 있습니다.

EMAIL joon@junu.dev

WEB junu.dev

GITHUB github.com/Vivacissimo

WIKI wiki.junu.dev

CONTENTS

01 **MoongCheap · Cloud Native Platform** — KT Cloud BootCamp

Frontend · Backend · AI 서비스를 AWS EKS에 일관되게 배포하기 위한 CI/CD 및 GitOps 구조 설계

02 **Hotel Booking · GitOps & Observability** — KT Cloud BootCamp

특가 트래픽이 몰리는 호텔 예약 MSA를 Terraform · GitHub Actions · ArgoCD로 EKS dev/prod에 자동 배포

03 **Medical MSA · CI/CD & Service Mesh** — KT Cloud BootCamp

의료 MSA의 CI/CD · GitOps 배포를 자동화하고, Istio로 Canary · Blue/Green · Retry · Zero-Trust를 검증

04 **Cocoavision · VFX Pipeline / Infrastructure** — Professional

DCC 환경과 Render Farm을 운영하며 Tractor 제출, PXE 설치, Ansible 노드 관리를 자동화

05 **junu.dev Home Infrastructure** — Personal

개인 서비스와 기술 위키를 직접 배포·운영하며 DNS, Container, Backup, CI/CD를 반복해서 경험하는 Home Lab

AWS EKS · MULTI-AZ · GITOPS · OBSERVABILITY

MoongCheap

소비자가 수요를 먼저 모으면 판매자가 가격으로 응찰하는
수요 집결형 AI 역경매 공동구매 플랫폼

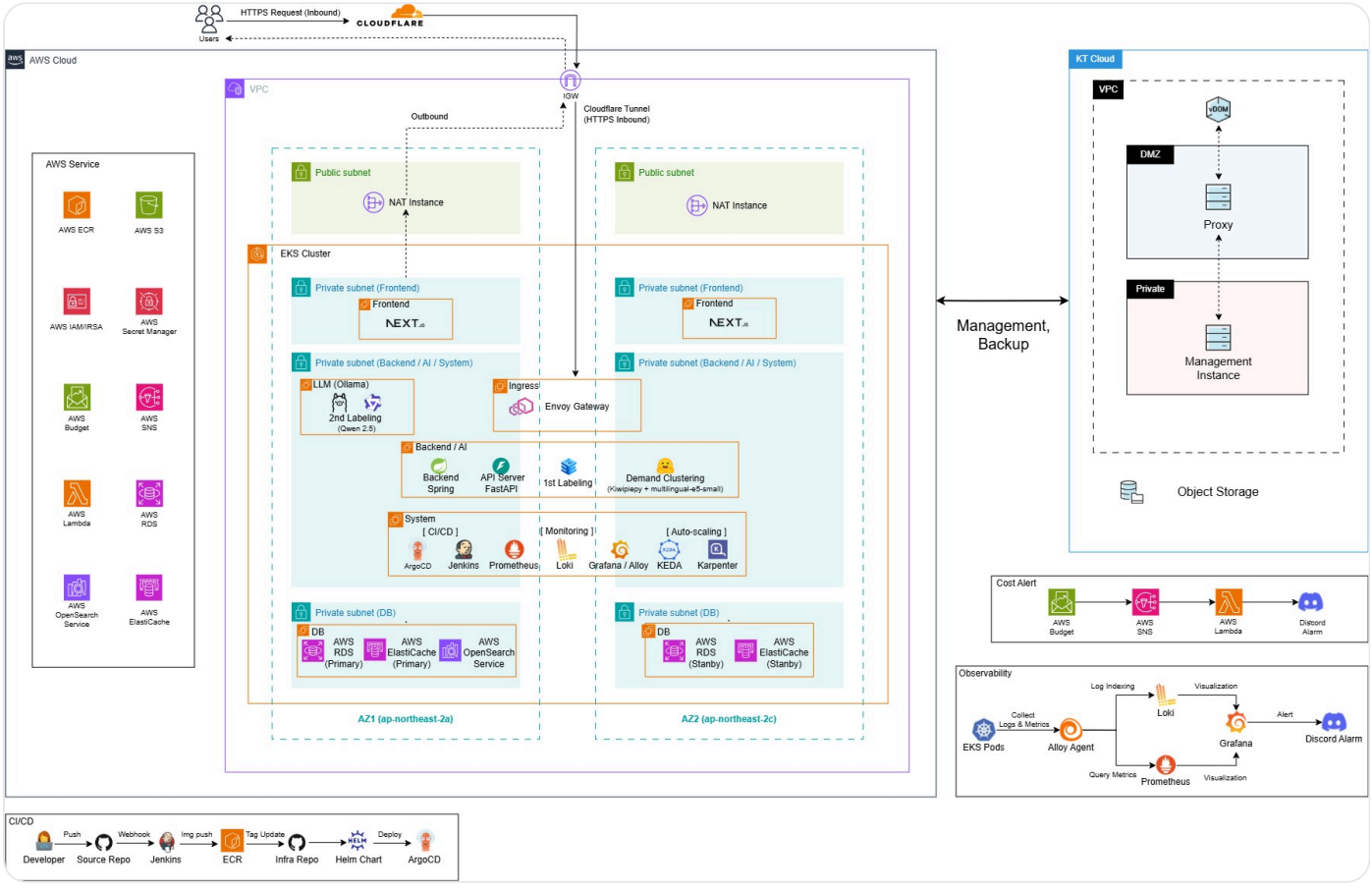
PERIOD	2026.07 ~ 2026.10
ROLE	CI/CD · Kubernetes · GitOps 담당
STACK	AWS EKS · Helm · ArgoCD · Jenkins · Prometheus · Loki · Grafana
REPO	github.com/V-MoongCheap/MoongCheap-Cloud ↗

01

프로젝트 개요

EKS Multi-AZ · Cloudflare Tunnel · Envoy Gateway · KEDA · Karpenter · RDS · ElastiCache Primary / Standby · Prometheus · Loki · Grafana · Cost Alert (Budget → SNS → Lambda) · KT Cloud 백업

SYSTEM ARCHITECTURE

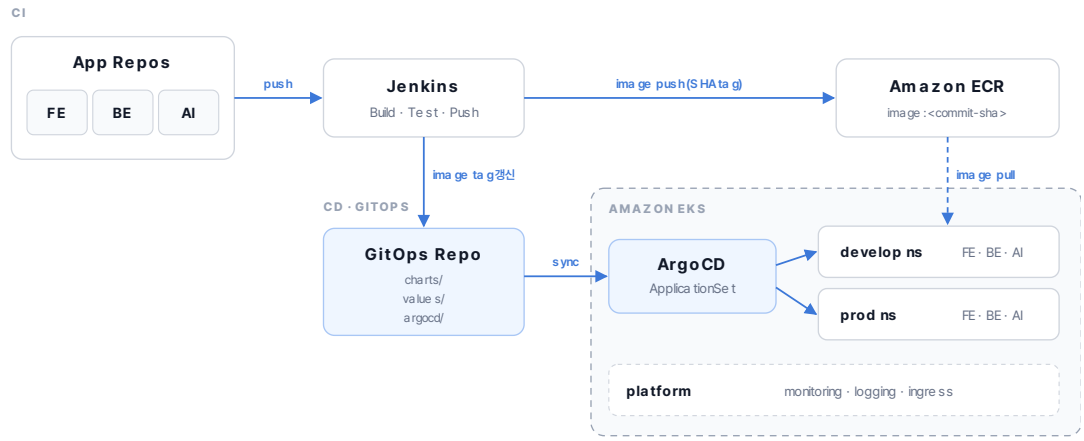


02

내 역할

Cloud 파트 5인 중 **CI/CD 전체**(서비스 컨테이너화 · Jenkins CI · ECR · GitOps · ArgoCD 배포)와 **AI · LLM 서비스 배포**를 맡았다.

MY SCOPE · CI/CD · GITOPS



HELM

공통 Chart + Values Layering

Frontend · Backend · AI가 **moongcheap-service** Chart 1개를 쓰고, 차이는 **base → env → services → overrides** 4단계 Values로만 관리한다.

ARGOCD

ApplicationSet으로 배포 자동 생성

develop / prod / platform ApplicationSet이 서비스별 Application을 생성하고, AppProject로 Service와 Platform의 배포 권한을 나눴다.

CI/CD

Commit SHA 기반 Image Tag

Jenkins가 SHA 태그로 ECR에 Push하고 GitOps Repo의 Tag를 갱신해, 운영 중인 Pod가 어떤 Commit인지 바로 추적할 수 있다.

GITOPS

App Repo / GitOps Repo 분리

코드 변경과 배포 상태 변경의 리뷰를 분리하고, 서비스 팀이 넘겨줄 항목(Port, Health Check, 환경 변수)을 정의해 협업 경계를 정했다.

AI · LLM

AI 서비스 배포 요구사항 정리

AI 팀에게 CPU · GPU · Memory 요구량, 추론 API, 이미지 크기, 모델 저장 위치를 받아 다른 서비스와 같은 공통 Chart · GitOps 흐름으로 EKS에 배포했다.

03

트러블슈팅

CI/CD를 운영하며 생긴 문제를 노드를 늘리거나 장비를 바꾸지 않고, 로그와 실측 데이터로 원인을 찾아 추가 비용 없이 해결했다.

Jenkins 기동 지연 — 96분 이상 → 2.6분

증상	평일 Open 스케줄로 인프라를 켠 뒤 CI 서버가 1시간 반 넘게 열리지 않아, 그동안 빌드를 할 수 없었다.
원인	**initializeOnce** 가 꺼져 있어 Jenkins가 기동할 때마다 플러그인 약 100개를 다시 설치했다.
해결	**initializeOnce: true** 설정 한 줄로 최초 1회만 설치하도록 변경.
결과	**기동 시간 96분 이상 → 2.6분 (97%↓)**

백엔드 빌드 시간 — 45분 → 16분

증상	백엔드 빌드 1회에 45분이 걸렸다.
원인	빌드 Pod의 메모리 요청값이 실제 사용량보다 커서, 빌드할 때마다 노드가 4번 추가되며 그만큼 스케줄링을 기다렸다.
해결	빌드 Pod 46개의 실제 리소스 사용량을 측정해 요청값을 다시 산정.
결과	**빌드 시간 45분 → 16분 (64%↓)**, 빌드당 노드 추가 4회 → 1회

노드당 Pod 한도 — 17개 → 110개

증상	노드 자원에 여유가 있는데도 ArgoCD · Jenkins Pod가 `Pending` 상태로 남았다.
원인	노드당 Pod 한도가 17개라 `Too many pods` 로 스케줄링이 거부됐다.
해결	VPC CNI Prefix Delegation을 켜고 `maxPods: 110` 으로 설정. 서비스 영향이 없도록 Close 시간에 적용했다.
결과	**노드당 최대 Pod 수 17개 → 110개 (6.5배)**, 노드 추가 없이 해결

04

회고

아쉬운 점

빌드 시간 단축을 리소스 조정에서 멈춤

요청값 조정으로 백엔드 빌드를 45분에서 16분으로 줄였지만, 캐시나 병렬화처럼 파이프라인 자체를 개선하는 단계까지는 가지 못했다.

다시 한다면

의존성과 이미지 레이어 캐시를 도입하고, 변경된 서비스만 빌드하도록 파이프라인을 나누겠다. 빌드 시간을 지표로 수집해 개선 효과를 계속 추적하겠다.

아쉬운 점

팀별 CI 파이프라인의 통일성 부족

CD는 공통 Helm Chart와 ApplicationSet 규칙으로 맞췄지만, CI는 Frontend · Backend · AI 팀마다 파이프라인 구성이 달라 단계와 규칙이 일관되지 않았다.

다시 한다면

Build · Scan(Trivy · gitleaks) · Push · Tag 갱신 단계를 공통 Jenkins 템플릿(Shared Library)으로 만들고, 각 팀은 언어별 빌드 설정만 넘기도록 해 CI도 CD와 같은 수준으로 표준화하겠다.

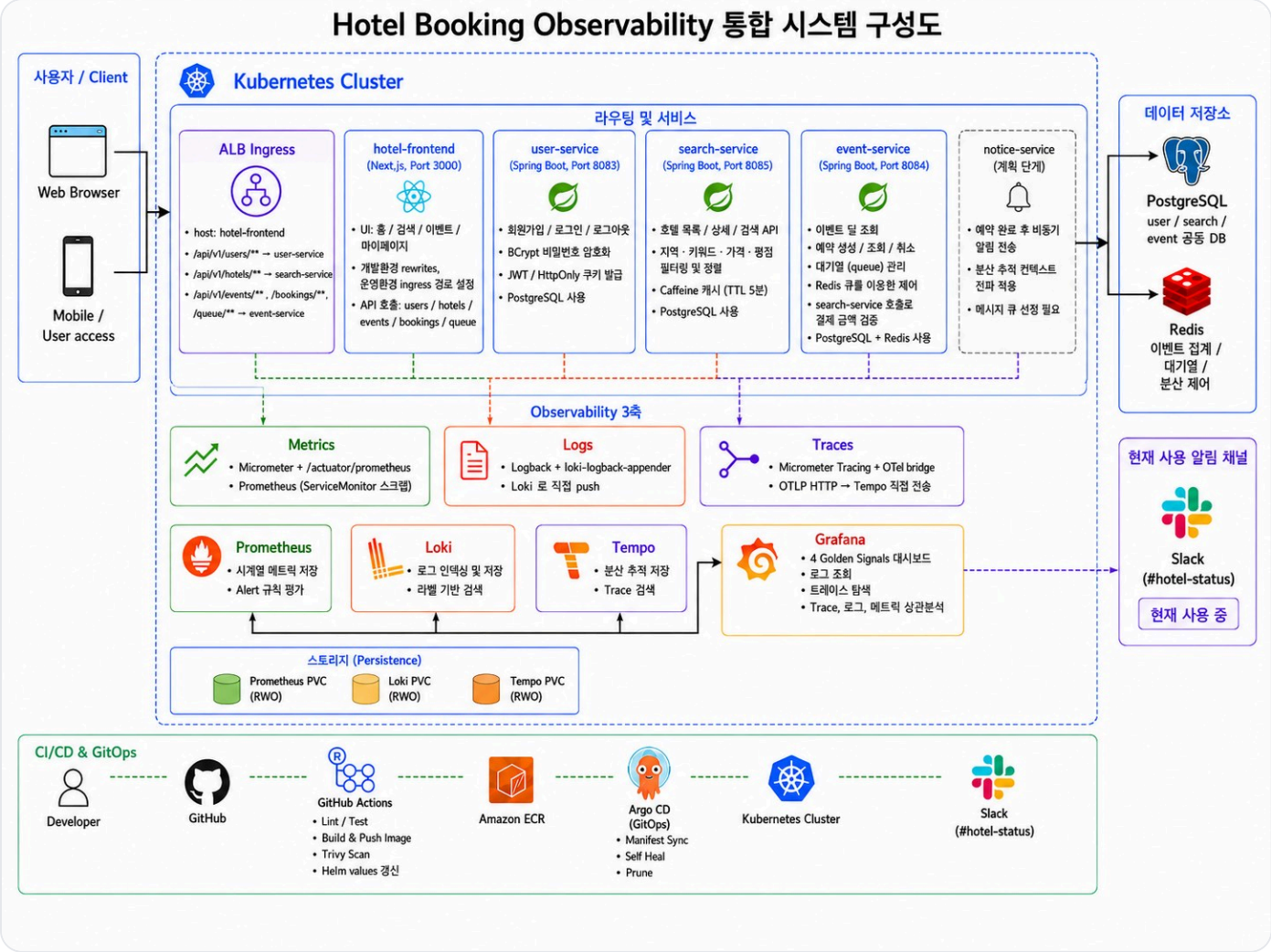
AWS EKS · TERRAFORM · GITHUB ACTIONS · ARGOCD · OBSERVABILITY

Hotel Booking · GitOps & Observability

특가 오픈 순간 트래픽이 몰리는 호텔 예약 MSA를 EKS에 배포하고, **push 한 번으로 dev/prod까지 이어지는 배포 흐름**을 만들었다.

PERIOD	2026.07 · KT Cloud BootCamp
TEAM	3인 (NoMoreSh)
ROLE	Team Lead · Infra & CI/CD
SERVICES	Frontend · User · Search · Event · Notice
PLATFORM	AWS EKS · Terraform · ArgoCD

SYSTEM ARCHITECTURE



01

프로젝트 개요

특가 오픈처럼 짧은 시간에 요청이 몰리는 상황에서도, 배포는 사람 손을 타지 않고 장애는 한 화면에서 찾을 수 있는 운영 환경을 목표로 했다.

Next.js 프론트엔드와 Spring Boot 4개 서비스(회원 · 검색 · 예약 · 알림)로 구성된 호텔 예약 MSA를 AWS EKS에 올렸다. 팀장이자 인프라 · CI/CD 담당으로 Terraform 기반 EKS 구성, GitHub Actions CI, Helm Chart, ArgoCD GitOps 배포와 dev/prod 환경 분리를 맡았다. 팀원은 대기열 · 재고 로직과 분산 추적(MSA 개발), 메트릭 · 로그 · 알림(Observability 스택)을 담당했다.

PROJECT GOAL

배포 자동화 + 3축 관찰성

Build → Scan → Deploy → Observe → Alert를 하나의 운영 흐름으로 연결했다.

MY ROLE

팀장 · Infra & CI/CD

클러스터부터 배포 파이프라인, 환경 분리까지 서비스가 올라가는 경로 전체를 담당했다.

ENVIRONMENT

dev / prod 분리

브랜치(`joon` / `main`), Namespace, Helm values 파일로 환경을 나눴다.

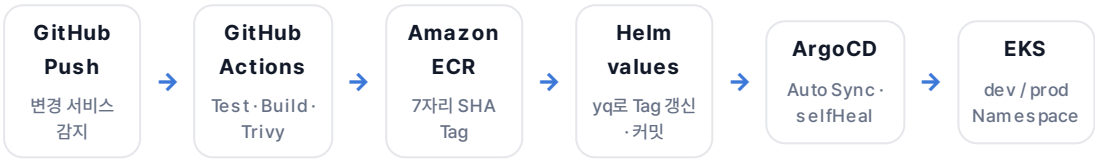
VERIFY

K6 1,000명 버스트 · 장애 주입

특가 트래픽과 서비스 중단 상황에서 정합성, 알림, 복구 흐름을 확인했다.

02

배포 흐름



03

내 역할

TERRAFORM

EKS 클러스터 코드화

`terraform-aws-modules/eks` 로 t3.medium 노드 그룹(3대, 1~4대)과 IRSA를 구성하고, Access Entry로 팀원 권한을 부여했다. ArgoCD도 Terraform Helm Release로 함께 설치했다.

CI

변경된 서비스만 빌드하는 Matrix

`paths-filter` 로 바뀐 서비스만 골라 Matrix를 만들고, 백엔드는 Postgres · Redis 서비스 컨테이너를 띄워 Gradle 테스트, 프론트엔드는 npm 빌드를 실행했다.

SECURITY

Trivy 스캔으로 Push 차단

이미지를 로컬에 먼저 빌드하고 HIGH · CRITICAL 취약점이 있으면 실패시켜, 검사를 통과한 이미지만 ECR에 올라가도록 했다.

GITOPS

SHA Tag → values 갱신 → 자동 Sync

CI가 이미지 Tag를 Helm values에 커밋하면 ArgoCD가 감지해 배포한다. App of Apps(`root-dev` / `root-prod`)와 prune · selfHeal로 클러스터를 Git 상태에 맞췄다.

PROD GUARD

prod 배포 전 매니페스트 검증

prod 파이프라인은 `helm lint` 와 렌더링 후 `namespace: dev` , `targetRevision: joon` , 빈 값이 섞여 있으면 실패하도록 해 dev 설정 유입을 막았다.

HELM

공통 기본값 + 서비스별 Override

단일 `hotel` Chart에서 `serviceDefaults` 를 서비스별 값과 병합했다. HPA(2~5, CPU 70%), 무중단 Rolling Update(`maxUnavailable: 0`), ALB Ingress 경로 라우팅을 공통으로 적용했다.

04

트러블슈팅

ALB 환경에서 로그인 후 401 반복

증상	EKS에 배포한 뒤 로그인 API는 성공하지만, 이후 인증이 필요한 요청에서 401이 반복됐다.
원인	JWT 쿠키에 `Secure` 속성이 붙어 있었는데 ALB가 HTTP로만 열려 있어, 브라우저가 쿠키를 저장하지 않았다.
해결	`Secure` 적용 여부를 환경 변수(`COOKIE_SECURE`)로 분리하고, 환경별 Helm values에서 값을 주입하도록 변경.
결과	**ALB를 통한 로그인 후 쿠키가 정상 저장되고, 마이페이지 · 예약 API가 401 없이 동작**

초기화 전 Liveness Probe로 Pod 반복 재시작

증상	배포 직후 Spring Boot Pod가 준비되기 전에 재시작을 반복했다.
원인	애플리케이션 초기화가 끝나기 전에 Liveness Probe가 실패로 판정했다.
해결	Chart 공통값에서 `initialDelaySeconds` 를 45초, `failureThreshold` 를 6으로 올려 초기화 시간을 확보.
결과	**시작 중 불필요한 재시작이 줄고, 배포 후 Pod가 안정적으로 Ready 상태에 도달**

클러스터에서 API 호출 실패 (CORS · 경로)

증상	로컬에서는 동작하던 프론트엔드 API 호출이 EKS 환경에서는 실패했다.
원인	서비스마다 CORS 설정이 흩어져 있었고, 프론트엔드가 로컬 서비스 주소를 직접 호출하고 있었다.
해결	CORS 설정을 공통화하고 API 호출을 상대경로로 바꾼 뒤, ALB Ingress에서 `/api/v1/*` 경로별로 서비스에 라우팅.
결과	**대기열(`/api/v1/queue`)을 포함한 API가 Ingress를 통해 정상 전달되어, 로컬과 클러스터 양쪽에서 동작**

05

검증 결과

배포한 환경 위에서 특가 트래픽과 장애 상황을 실제로 재현해 확인했다.

재고 30개 특가에 1,000명이 동시에 구매하는 상황을 K6로 재현했다. 대기열 슬롯 미반납, 토큰 발급 경쟁 상태 같은 병목은 팀원이 수정했고, 최종 테스트에서 정합성 목표는 충족했다. 다만 전체 요청 p95는 239ms로 목표(200ms)에 닿지 못했다. 장애 주입은 event-service Replica를 0으로 줄여 Slack ServiceDown 알림과 복구 후 RESOLVED 알림을 확인했고, ArgoCD에서 `hotel-prod` 가 Synced · Healthy로 유지되는 것을 확인했다.

LOAD TEST
오버셀 0건
1,000 VU · 재고 30 · checks 100%

ERROR
시스템 오류 0%
매진(409)은 정상 응답으로 분리

DEPLOY
5개 서비스 · 2개 환경
push 한 번으로 dev / prod 자동 배포

06

회고

아쉬운 점

클러스터에서만 생기는 로그 · 트레이스 유실을 끝까지 좁히지 못함

로컬에서는 메트릭 · 로그 · 트레이스가 모두 연결됐지만, EKS에서는 일부 서비스 로그가 Loki로 전송되지 않고 일부 POST 요청의 Span이 Tempo에 남지 않은 채로 마무리했다.

다시 한다면

배포 후 검증 단계에 로그 · 트레이스 수집 여부 확인을 넣어, 환경 차이를 배포 시점에 바로 잡겠다.

아쉬운 점

버스트 트래픽의 응답시간 목표 미달

정합성은 지켰지만 1,000명이 동시에 몰리는 순간의 p95는 239ms로 목표를 넘었다. HPA는 CPU 기준이라 짧은 버스트에 대응하는 확장 전략은 검증하지 못했다.

다시 한다면

대기열 길이나 요청량 같은 지표 기반 스케일링과 특가 오픈 전 사전 확장을 함께 검토하겠다.

아쉬운 점

dev / prod 워크플로 중복

두 워크플로가 대부분 같은 단계를 복사해 쓰고 있어, prod에만 있는 검증 단계처럼 환경 간 차이가 생기기 쉬웠다. AWS 인증도 Access Key를 Secret으로 넣는 방식이었다.

다시 한다면

Reusable Workflow로 공통 단계를 한 곳에 두고 환경별 값만 넘기며, AWS 인증은 GitHub OIDC로 바꿔 장기 키를 없애겠다.

Tech Stack: AWS EKS · Terraform · GitHub Actions · Amazon ECR · Trivy · Helm · ArgoCD · ALB Ingress · Prometheus · Loki · Tempo · Grafana · Alertmanager · Slack · K6

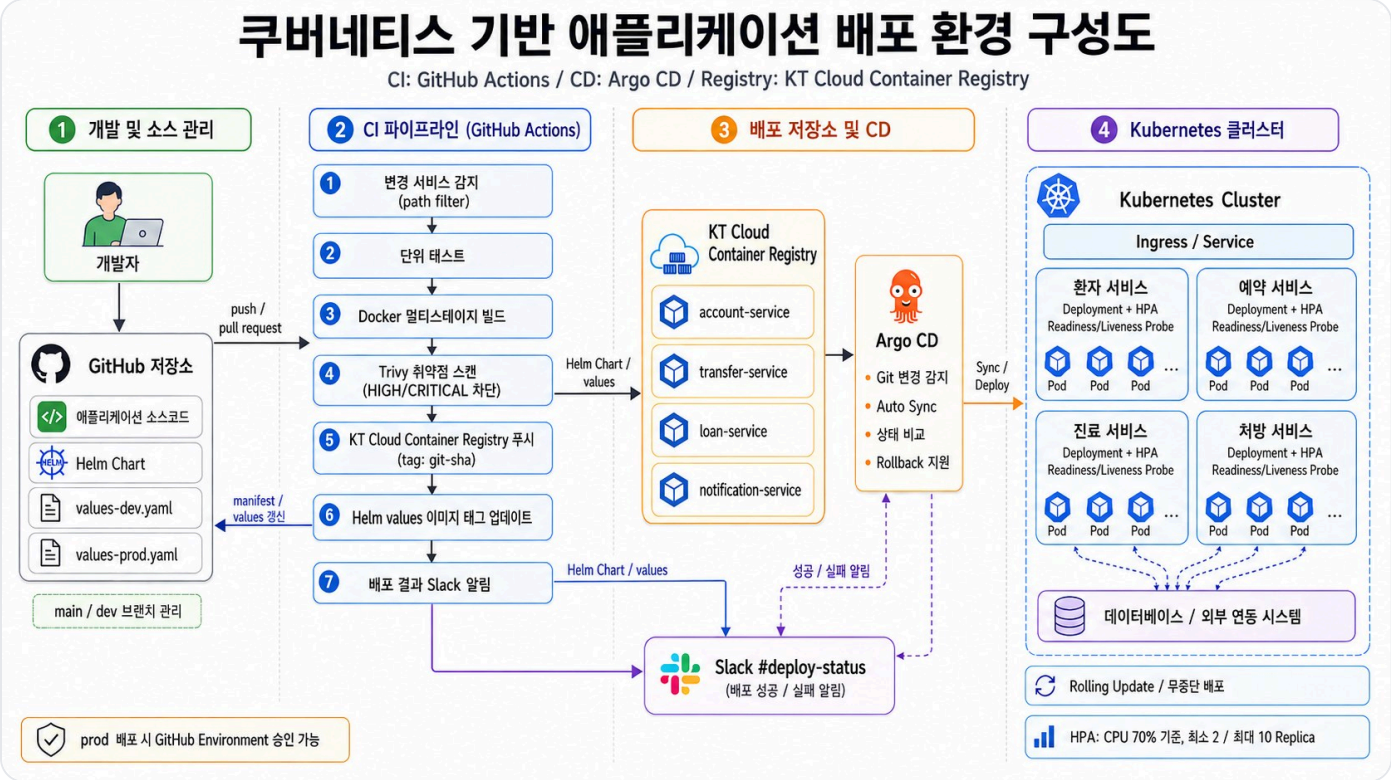
CI/CD · GITOPS · SERVICE MESH · OBSERVABILITY

Medical MSA · CI/CD & Service Mesh

의료 MSA의 **CI/CD · GitOps 배포 자동화**를 만들고, 심화 단계에서 같은 서비스에 Istio를 올려 **트래픽 전환 · 장애 복구 · 내부 접근 차단**을 검증했다.

PERIOD	2026.05 – 06 · KT Cloud BootCamp
TEAM	4인 (NoMoreSh)
ROLE	기본 CI/CD · GitOps · K8s 운영 · 심화 Team Lead · Observability
SERVICES	Patient · Appointment · Prescription · Notification
PLATFORM	Kubernetes · ArgoCD · AWS EKS · Istio

DEPLOYMENT ARCHITECTURE



01

프로젝트 개요

수동 배포에서 생기는 휴먼 에러와 서비스 중단을 줄이기 위해 배포를 자동화하고, 이어서 배포된 새 버전이 실제 트래픽과 장애 상황에서도 정상인지까지 확인하는 흐름을 만들었다.

환자 · 예약 · 처방 · 알림 서비스와 API Gateway로 구성된 의료 MSA를 대상으로 두 단계에 걸쳐 진행했다. 기본 단계(5주)에서는 GitHub Actions 기반 CI, 멀티스테이지 이미지 빌드, Trivy 보안 검사, 레지스트리와 Helm values 갱신부터 ArgoCD GitOps 배포, HPA · Probe · Rolling Update 같은 Kubernetes 운영 설정까지 배포 경로 전체를 맡았다. 심화 단계에서는 팀장이자 Observability 담당으로 AWS EKS 위에 Istio를 적용해 Canary · Blue/Green · Retry · Zero-Trust를 실험하고, Prometheus · Grafana · Kiali 관찰 환경과 장애 추적 순서를 정리했다.

PHASE 1 · 기본

CI/CD · GitOps 배포 자동화

push 한 번으로 변경 서비스만 테스트 · 빌드 · 검사해 dev/prod에 배포하고 결과를 Slack으로 공유했다.

PHASE 2 · 심화

Service Mesh · Observability

Deploy → Traffic Shift → Observe → Recover를 하나의 운영 절차로 연결했다.

WHY ISTIO

Istio vs Linkerd 비교(ADR)

L7 가중치 제어, mTLS · AuthorizationPolicy, Kiali · Prometheus 연동을 근거로 Istio를 선택했다.

CLUSTER

EKS 1.34 · t3.medium × 3

심화 단계는 노드 1~4대 범위로 운영하고, Pod마다 붙는 Envoy(100m / 128Mi) 비용을 함께 산정했다.

02 배포 흐름

GitHub Actions
Path Filter · Gradle Test
→
Image Build
Multi-stage · Buildx
→
Trivy
HIGH · CRITICAL 차단
→
Registry
git-sha Tag Push
→
Helm values
Tag 갱신 · Commit
→
ArgoCD
dev / prod Sync · Slack

03 내 역할

기본 단계 · CI/CD · GitOps · Kubernetes 운영

CI

변경된 서비스만 빌드하는 Matrix

서비스별 Path Filter로 수정된 서비스만 골라 Matrix로 빌드하고, Gradle 테스트가 실패하면 그 자리에서 파이프라인을 멈췄다.

IMAGE

멀티스테이지 빌드와 경량화

빌드는 `eclipse-temurin:17-jdk` , 실행은 `17-jre` 로 나눠 실행 이미지를 줄이고, 컨테이너를 비루트 사용자로 실행했다.

SECURITY

Trivy로 레지스트리 유입 차단

HIGH · CRITICAL 취약점이 있으면 `exit-code 1` 로 실패시켜, 검사를 통과한 이미지만 레지스트리에 올라가도록 했다.

REGISTRY

git-sha Tag Push와 values 갱신

이미지를 git-sha로 태깅해 레지스트리에 Push하고, Helm values의 이미지 태그를 갱신 · 커밋해 ArgoCD 배포로 이어지게 했다.

NOTIFY

Slack 배포 결과 알림

변경 서비스 목록, 빌드 결과, Helm 갱신 결과, 커밋 · 배포자 · workflow 정보를 `#deploy-status` 로 보내 GitHub에 들어가지 않고도 배포 상태를 확인하게 했다.

INFRA

빌드 환경과 레지스트리 운영

QEMU · Docker Buildx로 멀티 플랫폼 빌드를 구성하고, 비용 지원 제한에 맞춰 레지스트리를 KT Cloud에서 NCP로 옮겼다.

심화 단계 · 팀장 · Observability

TEAM LEAD

기본 → 심화 단계 흐름 정리

CI/CD, Service Mesh, Observability가 따로 놀지 않도록 실험 순서와 단계별 검증 포인트를 연결했다.

METRICS

Prometheus 수집 대상 연동

Kubernetes 컴포넌트와 Istio · Envoy 메트릭을 수집 대상으로 연결하고, Grafana에서 서비스별 요청량과 사이드카 리소스 점유율을 보도록 구성했다.

TOPOLOGY

Kiali 서비스 그래프

Ingress Gateway부터 각 서비스까지 호출 관계를 Mesh 관점으로 그려, 실패 지점과 버전별 트래픽 흐름을 확인했다.

RUNBOOK

3단계 장애 추적 순서

Grafana로 요청량 · 5xx · 리소스 수치 확인 → Kiali로 호출 단절 구간 확인 → ArgoCD로 배포 상태와 Git 이력을 대조해 복구하는 순서를 정했다.

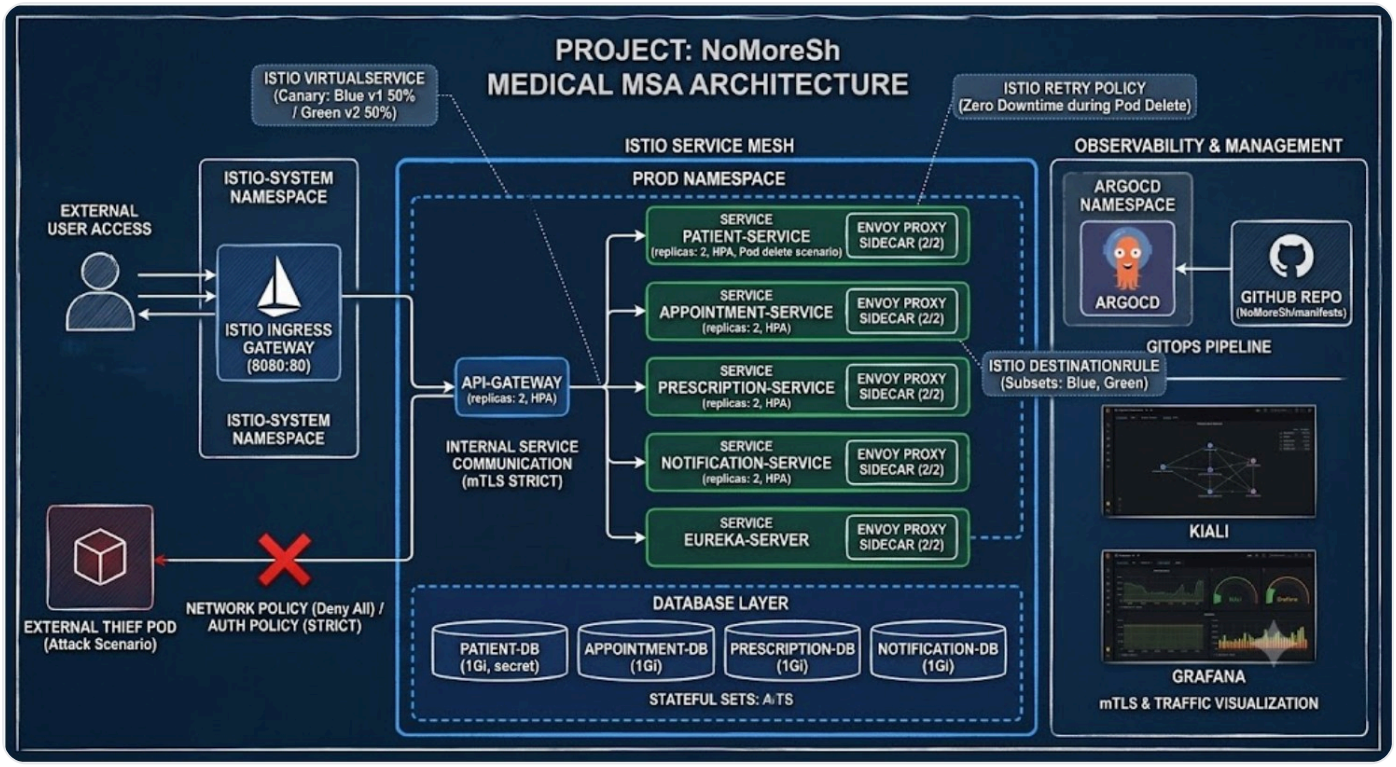
04

GitOps · K8s 운영

이미지가 올라간 뒤 클러스터에 배포되고, 배포 중과 장애 상황에서도 서비스가 유지되도록 설정하고 직접 검증했다.

GitOps	**하나의 Chart, 두 개의 환경** `charts/medical` 하나에 `values-dev` · `values-prod` 를 두고 ArgoCD Application(`medical-dev` / `medical-prod`)을 나눠, dev에서 먼저 배포를 검증하고 환경별 Sync · Health를 따로 확인했다.
HPA	**CPU 70% 기준 2 ~ 10 Replica** 최소 2개 Pod를 유지하고, 부하가 늘면 자동으로 늘렸다가 줄어들면 다시 축소한다.
Probe	**Readiness** `/health/ready` **· Liveness** `/health` 준비된 Pod만 트래픽을 받고, 비정상 Pod는 자동으로 재시작된다. DB 장애와 Memory Leak을 일부러 일으켜 Probe 실패(HTTP 500)를 유도하고, 실패 이벤트를 cron 스크립트로 수집해 Pod별 로그로 남겼다.
Rolling Update	`maxSurge: 1` · `maxUnavailable: 0` 새 Pod가 준비된 뒤에 기존 Pod를 내려 배포 중 트래픽이 끊기지 않게 했다.

SERVICE MESH ARCHITECTURE · 심화



05

Service Mesh 검증

Canary	**20% → 50% → 70% → 100%** VirtualService 가중치만 바꿔 새 버전 트래픽을 단계적으로 늘렸다. curl을 반복 호출해 지정한 비율대로 요청이 나뉘는 것을 로그로 확인했다.
Blue / Green	**50:50 → 0:100 즉시 전환** 두 버전을 띄워 둔 상태에서 라우팅 대상을 한 번에 바꿨다. 클라이언트 쪽 단절 없이 응답이 Blue에서 Green으로 바뀌는 것을 확인했다.
Retry	**Pod 강제 종료 중 클라이언트 성공 유지** VirtualService에 `retries.attempts: 3` 을 넣고, 요청을 보내는 도중 `kubectl delete pod` 로 Pod를 종료했다. Envoy가 503을 받고 살아 있는 Pod로 재시도해 클라이언트 요청은 모두 성공했다.
Zero-Trust	**비인가 Pod의 내부 접근 차단** mTLS STRICT와 AuthorizationPolicy를 적용한 상태에서 Mesh 밖의 Pod를 띄워 백엔드 주소로 직접 요청했다. `RBAC: access denied` 로 차단되는 것을 확인했고, Canary 경로로 다시 시도해도 같은 결과였다.

06

트러블슈팅

빌드 아키텍처 차이로 인한 배포 에러

증상	CI에서 만든 이미지가 배포 환경에서 실행되지 않아 배포가 실패했다.
원인	이미지를 빌드한 환경과 실행 환경의 CPU 아키텍처가 달랐다.
해결	QEMU와 Docker Buildx로 멀티 플랫폼 빌드 파이프라인을 구성해, 실행 환경에 맞는 이미지를 CI에서 바로 만들도록 변경.

openjdk 베이스 이미지 지원 종료

증상	Dockerfile이 쓰던 `openjdk` 공식 이미지가 Docker Hub에서 지원 종료돼, 보안 업데이트를 더 받을 수 없게 됐다.
해결	베이스 이미지를 `eclipse-temurin` 으로 교체하고, 이 기회에 jdk(빌드) / jre(실행) 멀티스테이지 구조와 비루트 실행을 함께 적용.

비용 제한으로 레지스트리 이전

증상	서버 비용 지원이 제한되면서 KT Cloud Container Registry를 계속 쓸 수 없게 됐다.
해결	레지스트리를 Naver Cloud Platform으로 옮기고, CI의 Push 단계를 NCP Registry 기준으로 바꿔 배포 흐름을 그대로 유지.

메트릭 수집 중 DNS · 네트워크 timeout (심화)

증상	Kubernetes와 Istio · Envoy 메트릭을 Prometheus 수집 대상으로 연결하는 과정에서 DNS · 네트워크 timeout이 발생했다.
추적	Prometheus 수집 상태와 Kiali의 서비스 간 호출 관계 · 에러 지점을 함께 보며 문제 구간을 좁혔다.
결과	**수집 대상 연동을 정상화하고, Grafana에서 서비스 · 사이드카 지표를 확인할 수 있게 됨**

운영 중 클러스터 리소스 고갈 (심화)

증상	실험을 진행하는 중 클러스터의 컴퓨팅 자원이 고갈됐다.
원인	프로젝트에 필요한 컴퓨팅 리소스 총량을 사전에 정확히 예측하지 못했다.
해결	노드 그룹 범위(1~4대) 안에서 운영 중 노드를 Scale-out해 자원을 재분배.
결과	**운영 중 노드를 늘려 자원을 다시 분배하는 동적 확장을 직접 검증하고 실험을 이어 감**

07

검증 결과

배포 자동화부터 트래픽 전환 · 장애 복구 · 접근 차단까지 실제 시나리오로 재현하고 확인했다.

기본 단계에서는 직접 만든 파이프라인으로 4개 서비스가 push 한 번으로 테스트 · 검사를 거쳐 배포되고, ArgoCD에서 Synced · Healthy로 유지되는 것을 확인했다. 심화 단계에서는 "배포 완료"가 아니라 "새 버전이 트래픽을 정상 처리하는가", "장애가 나도 사용자는 실패를 보지 않는가"까지 확인해야 운영이 끝난다는 기준을 팀의 Runbook으로 정리했다.

CI/CD

4개 서비스 자동 배포

Trivy 통과 이미지만 dev / prod 반영

RETRY

클라이언트 성공률 100%

Pod 강제 종료 중 3회 재시도

ZERO-TRUST

비인가 접근 차단

mTLS STRICT · AuthorizationPolicy

08

회고

아쉬운 점

기반 인프라를 셸 스크립트로 생성

배포 파이프라인과 트래픽 제어에 집중하느라 VPC · EKS 같은 기반 자원은 코드로 관리하지 못했다.

다시 한다면

Terraform으로 클러스터 바깥 인프라까지 선언형으로 관리하겠다. 다음 프로젝트(Hotel Booking)에서 EKS와 ArgoCD를 Terraform으로 구성하며 이 부분을 이어서 해결했다.

아쉬운 점

리소스 산정과 권한 설정의 시행착오

비용 제한으로 레지스트리를 옮기고, 심화 단계에서는 프리티어 자원 제약과 IAM 사용자 권한 매핑 문제로 클러스터 생성과 접근 권한 확보가 반복해서 지연됐다.

다시 한다면

사이드카 비용을 포함한 리소스 총량과 비용을 먼저 산정하고, 팀원별 클러스터 접근 권한을 생성 시점에 코드로 함께 정의하겠다.

Tech Stack: GitHub Actions · Docker Buildx · QEMU · Trivy · NCP Container Registry · Helm · ArgoCD · Kubernetes HPA · AWS EKS · Istio · Envoy · Prometheus · Grafana · Kiali · Slack

PROFESSIONAL EXPERIENCE · VFX PIPELINE / INFRASTRUCTURE

Cocoavision

작업자가 사용하는 DCC 환경과 Render Farm을
운영하며 반복 작업을 자동화하고 제작 중단 원인을 해결

PERIOD	2022.09 - 2026.03
ROLE	Pipeline / IT Infrastructure
USERS	Artist · Production · External Vendor
ENV	Linux · DCC · Render Farm
FOCUS	Troubleshooting · Automation · Standardization

01

업무 개요

사용자 문의를 한 번 해결하는 데서 끝내지 않고, 같은 문제가 반복될 때는 Pipeline이나 Infrastructure 차원에서 줄이는 방식으로 접근했다.

Houdini, Nuke, Maya 등 제작 도구와 Linux 기반 Render Farm 사이에서 작업이 안정적으로 이어지도록 환경을 운영했다. Render 실패, 경로/마운트 문제, 플러그인/버전 차이 등 제작 중 발생하는 이슈를 분석했고, 반복되는 설치와 설정 업무는 PXE와 Ansible로 자동화했다. 외주사와의 작업에서는 파일, 환경, 전달 방식의 차이에서 발생하는 문제를 기술적으로 설명하고 대응했다.

OPERATE
Linux 제작 환경과 Render Farm
작업 중단을 만드는 환경 문제를 확인하고 복구했다.

AUTOMATE
PXE · Ansible · Python
반복 설치, 노드 설정, 제출 과정을 자동화 가능한 형태로 바꿨다.

TROUBLESHOOT
DCC · Plugin · Path · Mount
로그와 실행 환경 차이를 기준으로 문제 범위를 좁혔다.

COMMUNICATE
Artist / Production / Vendor
기술적 원인과 필요한 조치를 비개발 사용자도 이해할 수 있게 전달했다.

02

핵심 문제

PROBLEM 01
작업자마다 다른 제출 방식
Render Farm 제출 과정이 수동 절차에 의존하면 누락과 설정 차이가 발생하기 쉽다.

PROBLEM 02
노드별 반복 설정
수십 대 규모의 Render Node를 개별로 관리하면 동일 작업이 반복되고 설정 편차가 생긴다.

PROBLEM 03
PC 초기 설치 반복
USB 기반 설치는 장비가 늘어날수록 사람이 직접 해야 하는 단계가 증가한다.

03

Case 01 · Tractor Submit

AS-IS

- 작업자가 제출 정보를 직접 구성
- DCC마다 제출 절차가 달라 실수 가능
- Job 규칙을 사용자에게 의존

→

TO-BE

- Houdini / Nuke 안에서 Submit
- 작업 정보를 Tractor Job 형식으로 변환
- 공통 제출 규칙을 Pipeline Tool에 반영

내 역할

Custom Render / Submit Node 구성
DCC에서 필요한 Job 정보를 수집하고 Render Farm으로 전달하는 흐름을

만들었다.

판단	**사용자 교육보다 도구에 규칙을 넣는 방식** 작업자마다 절차를 기억하게 하기보다 제출 도구가 동일한 입력과 Job 형식을 만들도록 했다.
결과	**Submission Workflow 표준화** 수동 Job 생성 단계를 줄이고 DCC 안에서 일관된 제출 경로를 제공했다.

04

Case 02 · PXE

AS-IS

- USB로 장비별 OS 설치
- 작업자가 직접 초기 세팅 반복
- 설치 절차 차이 발생 가능

→

TO-BE

- Network Boot 기반 설치
- PXE + DHCP/HTTP + Kickstart
- 초기 설치 절차 표준화

내 역할	**PXE 부팅과 자동 설치 환경 구성** 네트워크 부팅부터 설치 파일 제공, Kickstart 기반 자동 설치까지 연결했다.
판단	**장비 수가 늘어날수록 반복 작업이 커지는 문제를 Infrastructure로 해결** 개별 작업 속도를 높이는 대신 사람이 개입해야 하는 단계를 줄였다.
결과	**USB 중심 설치 → 네트워크 기반 설치** PC 초기 세팅을 동일한 절차로 반복할 수 있는 기반을 만들었다.

05

Case 03 · Ansible

AS-IS

- Render Node마다 SSH 접속 후 설정
- 패키지/마운트/도구 배포 반복
- 노드별 설정 편차 가능

→

TO-BE

- Inventory로 대상 노드 그룹화
- Playbook으로 공통 작업 선언
- 동일한 설정을 여러 노드에 반복 적용

대상	**수십 대 규모 Linux Render Farm**
자동화	**패키지 설치 · Mount · 팀별 Tool 배포 등** 반복 운영 작업을 Playbook 단위로 정리했다.
결과	**서버별 수동 작업을 재사용 가능한 운영 코드로 전환** 동일 작업을 여러 노드에 일관되게 적용할 수 있게 했다.

06

장애 대응 사례

ISSUE

로컬에서는 정상인데 Farm에서 반복 실패

아티스트 작업이 로컬에서는 열리지만 Render Farm 제출 후 실패해 제작이 중단되는 상황.

ANALYSIS

실행 환경 차이를 기준으로 범위 축소

Tractor Log, DCC/Plugin 버전, 환경 변수, 파일 경로, Mount 상태를 순서대로 비교했다.

ACTION

설정 수정 + 반복 방지

문제 설정을 수정한 뒤 같은 유형이 다른 노드에 반복되지 않도록 운영 절차와 자동화 대상을 보완했다.

장애 대응에서 가장 중요하게 본 것은 "어디에서만 실패하는가"였다. 로컬과 Farm의 차이를 비교해 원인을 좁히고, 해결 후에는 같은 문제를 운영 코드나 절차로 줄이는 방식으로

접근했다.

07

성과와 연결점

VFX에서 경험한 운영 문제를 DevOps에서 사용하는 방식과 자연스럽게 연결할 수 있었다.

Render Farm 운영은 다수 노드의 일관된 설정, 장애 로그 분석, 반복 작업 자동화, 사용자와의 협업이 필요한 환경이었다. 이 경험은 이후 Kubernetes, CI/CD, IaC를 학습할 때 단순 도구 사용보다 운영 문제를 어떤 구조로 줄일지 생각하는 기반이 됐다.

FARM	AUTOMATION	WORK STYLE
수십 대 규모 노드	**3개 핵심 자동화 영역**	**Fix → Standardize → Automate**
Linux Render Node 운영	Submit · Provisioning · Configuration	반복 이슈를 구조적으로 줄이는 방식

Tech Stack: Linux · Houdini · Nuke · Maya · Tractor · Python · Ansible · PXE · Unreal Engine · Blender

PERSONAL INFRASTRUCTURE · SELF-HOSTED

junu.dev Home Infrastructure

Raspberry Pi 한 대에 기술 위키와 웹 도구를 올리고, **push 한 번으로 배포되고 포트를 열지 않고 공개되는** 개인 서비스 환경을 직접 운영한다.

PERIOD	운영 중
ROLE	Design · Deploy · Operate (1인)
BASE	Raspberry Pi · Linux · NVMe
EDGE	Cloudflare Tunnel · DNS
SERVICES	wiki · maskly · transformer .junu.dev

01 프로젝트 개요

실습용 VM을 만들고 지우는 대신, 실제로 매일 쓰는 서비스를 계속 운영하면서 배포 · 네트워크 · 데이터 문제를 직접 겪어 보기 위한 환경이다.

Raspberry Pi Linux 서버에서 Wiki.js 기술 위키와 개인 웹 도구를 운영한다. 위키는 Docker Compose로 Wiki.js와 PostgreSQL을 분리해 띄우고, 웹 도구는 Nginx가 정적 파일로 서비스한다. 외부 접근은 Cloudflare Tunnel로만 받아 집 공유기의 포트를 열지 않았고, 웹 도구는 GitHub Actions에서 빌드해 Pi에 설치한 Self-hosted Runner가 배포한다.

GOAL

실사용 서비스를 직접 운영

설치하고 끝나는 실습이 아니라, 바뀌고 고장 나는 서비스를 계속 유지한다.

EDGE

Origin을 공개하지 않는 노출

Cloudflare Tunnel이 바깥으로 연결을 맺어, 인바운드 포트 없이 서브도메인별로 서비스를 연결한다.

DELIVERY

빌드와 배포 Runner 분리

무거운 빌드는 GitHub-hosted Runner에서, 파일 반영만 Pi의 Self-hosted Runner에서 실행한다.

DATA

컨테이너와 데이터 분리

위키 데이터는 PostgreSQL 볼륨과 SQL Dump로 관리해 컨테이너를 다시 만들어도 유지된다.

02 배포 흐름

GitHub Push (master 브랜치) → Build Job (npm ci · lint · build) → Artifact (dist 업로드) → Pi Runner (Self-hosted · Label 지정) → Nginx (rsync --delete 반영) → Cloudflare Tunnel (*.junu.dev 공개)

03 내 역할

CI/CD

GitHub Actions 2단계 파이프라인

Build Job이 lint와 build를 통과한 결과물만 Artifact로 올리고, Deploy Job이 이를 받아 반영한다. lint가 실패하면 배포까지 가지 않는다.

RUNNER

Self-hosted Runner 라벨 라우팅

Pi에 Runner를 설치하고 `raspberrypi` · `deploy` · 서비스 이름 라벨을 붙여, 배포 Job이 정해진 장비에서만 실행되게 했다.

RELEASE

동기화 방식의 정적 배포

`rsync --delete` 로 서비스 디렉터리를 빌드 결과와 똑같이 맞춰 이전 파일이 남지 않게 하고, 배포 후 디렉터리 755 · 파일 644로 권한을 정리한다.

CONCURRENCY

중복 배포 방지

워크플로에 `concurrency` 그룹을 두어, 연속으로 push해도 마지막 변경만 배포되도록 이전 실행을 취소한다.

CONTAINER

Docker Compose 위키 구성

Wiki.js와 PostgreSQL을 컨테이너로 분리하고 볼륨으로 데이터를 유지하며, SQL Dump로 백업한다.

HARDWARE

microSD → NVMe 이전

계속 쓰고 지우는 서버 특성상 수명과 속도가 약한 microSD를 NVMe로 바꿔 저장장치를 안정화했다.

04

Personal Dev Tools

개인 작업에서 반복되던 일을 AI 에이전트와 함께 작은 웹 도구로 개발하고, 위 배포 흐름과 Cloudflare Tunnel로 각자 서브도메인에서 운영한다.

MASKLY · maskly.junu.dev

웹 기반 이미지 편집기

React · Vite로 만든 레이어 기반 편집기. 레이어 마스크, 브러시, 알파 편집, 실행 취소를 브라우저에서 처리한다.

TRANSFORMER · transformer.junu.dev

브라우저 파일 변환기

이미지 포맷 · 품질 · 크기 변환, 이미지/텍스트 → PDF, TXT · JSON · CSV · HTML · Markdown 변환을 서버 없이 브라우저에서 처리한다.

05

결과

코드를 push하면 빌드 · 검사를 거쳐 Pi에 자동 반영되고, 집 네트워크에 포트를 열지 않은 채 서비스가 공개된다.

배포가 끝난 뒤에도 DNS, Tunnel 연결, 컨테이너 재시작, 데이터 백업처럼 서비스를 살려 두기 위해 확인해야 하는 일을 직접 반복하고 있다. 부트캠프에서 다룬 Self-hosted Runner, 빌드 · 배포 분리, 데이터 볼륨 분리를 작은 규모로 계속 써 보는 환경이기도 하다.

DELIVERY

push → 자동 배포

Build(GitHub-hosted) · Deploy(Pi) 분리

EDGE

인바운드 포트 없이 공개

Cloudflare Tunnel · 서브도메인별 연결

SERVICES

위키 + 웹 도구 2개

wiki · maskly · transformer

Tech Stack: Linux · Raspberry Pi · Docker Compose · Nginx · Cloudflare Tunnel · GitHub Actions · Self-hosted Runner · PostgreSQL · Wiki.js · React · Vite · AI Agent